

Condicionales con 10000

Pensamiento Computacional

Facultad de Ciencias Exactas y Naturales, UBA

Segundo Cuatrimestre 2026

ACTIVIDAD DE HOY

Diez mil: ¿lo conoces?

Una excusa para romper el hielo



- ¿Cómo se juega?
- ¿Jugaste? ¿Dónde? ¿Cuándo?
- ¿Ganaste?

Diez mil:



- Es un juego **popular de dados**, ideal para días de lluvia.

Diez mil:



- Es un juego **popular de dados**, ideal para días de lluvia.
- Se juega entre **varios jugadores** (más de dos).

Diez mil:



- Es un juego **popular de dados**, ideal para días de lluvia.
- Se juega entre **varios jugadores** (más de dos).
- Se usan **cinco dados**.

Diez mil:



- Es un juego **popular de dados**, ideal para días de lluvia.
- Se juega entre **varios jugadores** (más de dos).
- Se usan **cinco dados**.
- Cada jugador va **tirando** los dados y **sumando** el puntaje que estos indican.

Diez mil:



- Es un juego **popular de dados**, ideal para días de lluvia.
- Se juega entre **varios jugadores** (más de dos).
- Se usan **cinco dados**.
- Cada jugador va **tirando** los dados y **sumando** el puntaje que estos indican.
- El objetivo es llegar a **10.000 puntos**.

Diez mil:



- Es un juego **popular de dados**, ideal para días de lluvia.
- Se juega entre **varios jugadores** (más de dos).
- Se usan **cinco dados**.
- Cada jugador va **tirando** los dados y **sumando** el puntaje que estos indican.
- El objetivo es llegar a **10.000 puntos**.

Diez mil: versión “simplificada” (por no decir otro juego...)

Vamos a tomar el *espíritu* del 10.000 para armar un juego que podamos implementar con un programa, así que las reglas van a estar **relajadas** (ya las vamos a ir viendo).

Puntajes: nuestras reglas

- Los **únicos** resultados que **suman puntos** son los *unos* y los *cincos*. El resto no aporta.

Puntajes: nuestras reglas

- Los **únicos** resultados que **suman puntos** son los *unos* y los *cincos*. El resto no aporta.
- Según **cuántos** obtengamos de cada uno, tendremos **diferentes** cantidades de **puntos**

Puntajes: nuestras reglas

- Los **únicos** resultados que **suman puntos** son los *unos* y los *cinco*s. El resto no aporta.
- Según **cuántos** obtengamos de cada uno, tendremos **diferentes** cantidades de **puntos**
- Por cada *uno* obtenido, se suman **100 puntos** y por cada *cinco*, **50**

Puntajes: nuestras reglas

- Los **únicos** resultados que **suman puntos** son los **unos** y los **cinco**. El resto no aporta.
- Según **cuántos** obtengamos de cada uno, tendremos **diferentes** cantidades de **puntos**
- Por cada **uno** obtenido, se suman **100 puntos** y por cada **cinco**, **50**
- Además, tenemos **combos** que nos hacen **sumar más puntos**. **No se pueden combinar** para un mismo número. **Sí** se puede usar **un combo** para los **unos** y **otro** para los **cinco**s, pero no más de uno para cada uno.

Puntajes: nuestras reglas

- Los **únicos** resultados que **suman puntos** son los **unos** y los **cinco**. El resto no aporta.
- Según **cuántos** obtengamos de cada uno, tendremos **diferentes** cantidades de **puntos**
- Por cada **uno** obtenido, se suman **100 puntos** y por cada **cinco**, **50**
- Además, tenemos **combos** que nos hacen **sumar más puntos**. **No se pueden combinar** para un mismo número. **Sí** se puede usar **un combo** para los **unos** y **otro** para los **cinco**s, pero no más de uno para cada uno.
- **3 unos** dan adicionalmente **1000 puntos**

Puntajes: nuestras reglas

- Los **únicos** resultados que **suman puntos** son los **unos** y los **cinco**. El resto no aporta.
- Según **cuántos** obtengamos de cada uno, tendremos **diferentes** cantidades de **puntos**
- Por cada **uno** obtenido, se suman **100 puntos** y por cada **cinco**, **50**
- Además, tenemos **combos** que nos hacen **sumar más puntos**. **No se pueden combinar** para un mismo número. **Sí** se puede usar **un combo** para los **unos** y **otro** para los **cinco**s, pero no más de uno para cada uno.
- **3 unos** dan adicionalmente **1000 puntos**
- **5 unos** dan adicionalmente **¡10.000 puntos!**

Puntajes: nuestras reglas

- Los **únicos** resultados que **suman puntos** son los **unos** y los **cincos**. El resto no aporta.
- Según **cuántos** obtengamos de cada uno, tendremos **diferentes** cantidades de **puntos**
- Por cada **uno** obtenido, se suman **100 puntos** y por cada **cinco**, **50**
- Además, tenemos **combos** que nos hacen **sumar más puntos**. **No se pueden combinar** para un mismo número. **Sí** se puede usar **un combo** para los **unos** y **otro** para los **cincos**, pero no más de uno para cada uno.
- **3 unos** dan adicionalmente **1000 puntos**
- **5 unos** dan adicionalmente **¡10.000 puntos!**
- Para las mismas cantidades de **cincos**, se obtienen adicionalmente **500** y **5000** puntos, respectivamente.

Puntajes: ejemplitos



- Total

Puntajes: ejemplitos



- Total: **0 puntos** (no hay *unos* ni *cincos*).

Puntajes: ejemplitos



Puntajes: ejemplitos



- Fácil, **no** hay combos.
- Total: 150 (un *uno* (100) + un *cinco* (50)).

Puntajes: ejemplitos



Puntajes: ejemplitos



- Tampoco hay combos.
- Total: 300 (dos *unos* ($2 \cdot 100$) + dos *cincos* ($2 \cdot 50$)).

Puntajes: ejemplitos



Puntajes: ejemplitos



- Total: **350** (tres *unos* ($3 \cdot 100$) + un *cinco* ($1 \cdot 50$)).

Puntajes: ejemplitos



- Total: **350** (tres *unos* ($3 \cdot 100$) + un *cinco* ($1 \cdot 50$)).
- **¡Había combos!** Corregimos:
- Total: **1350** (tres *unos* ($3 \cdot 100$) + un *cinco* ($1 \cdot 50$) + **combo** de 3 *unos* (**1000**)).

Puntajes: ejemplitos



Puntajes: ejemplitos



- Total: **250** (un *uno* (100) + tres *cincos* (3*50)).

Puntajes: ejemplitos



- Total: **250** (un *uno* (100) + tres *cincos* (3*50)).
- **¡Había combos!** Corregimos:
- Total: **750** (un *uno* (100) + tres *cincos* (3*50) + **combo** de 3 *cincos* (500)).

Puntajes: ejemplitos



Puntajes: ejemplitos



- Total: **250** (cinco *cincos* (5×50)).

Puntajes: ejemplitos



- Total: **250** (cinco *cincos* ($5 \cdot 50$)).
- **¡Había combos!** Corregimos:
- Total: **5750** (cinco *cincos* ($5 \cdot 50$) + **combo** de 3 *cincos* (**500**) + **combo** de 5 *cincos* (**5000**)).

Puntajes: ejemplitos



- Total: **250** (cinco *cincos* ($5 \cdot 50$)).
- **¡Había combos!** Corregimos:
- Total: **5750** (cinco *cincos* ($5 \cdot 50$) + **combo** de 3 *cincos* (**500**) + **combo** de 5 *cincos* (**5000**)).
- **¡Solo combo de mayor valor!** Corregimos:
- Total: **5250** (cinco *cincos* ($5 \cdot 50$) + **combo** de 5 *cincos* (**5000**)).

EL JUEGO

¿Cómo se juega?

- Cada jugador **comienza con 0 puntos**.

¿Cómo se juega?

- Cada jugador **comienza con 0 puntos**.
- En una **ronda** cada jugador realiza una **jugada**.

¿Cómo se juega?

- Cada jugador **comienza con 0 puntos**.
- En una **ronda** cada jugador realiza una **jugada**.
- En cada **jugada** se lanza el cubilete con cinco dados.

¿Cómo se juega?

- Cada jugador **comienza con 0 puntos**.
- En una **ronda** cada jugador realiza una **jugada**.
- En cada **jugada** se lanza el cubilete con cinco dados.
- Se calculan los puntajes según las **reglas que vimos recién**.

¿Cómo se juega?

- Cada jugador **comienza con 0 puntos**.
- En una **ronda** cada jugador realiza una **jugada**.
- En cada **jugada** se lanza el cubilete con cinco dados.
- Se calculan los puntajes según las **reglas que vimos recién**.
- Al finalizar la **ronda** se **suman** los **puntos** obtenidos por cada jugador **con los que ya tenía**.

¿Cómo se juega?

- Cada jugador **comienza con 0 puntos**.
- En una **ronda** cada jugador realiza una **jugada**.
- En cada **jugada** se lanza el cubilete con cinco dados.
- Se calculan los puntajes según las **reglas que vimos recién**.
- Al finalizar la **ronda** se **suman** los **puntos** obtenidos por cada jugador **con los que ya tenía**.
- El juego **termina** cuando, al finalizar una ronda, **algún jugador** alcanza (o supera) los 10.000 puntos, es decir, **gana el juego**.

¡A jugar!

JUGAMOS ~~10~~ MIL

Puntajes	
Cada 	suma 100 puntos
Cada 	suma 50 puntos
Puntos adicionales por combo :	
3 	= 1.000 puntos
5 	= 10.000 puntos
3 	= 500 puntos
5 	= 5.000 puntos
Entre todos los combos posibles, siempre se elije al de mayor puntaje.	
   	No dan puntos

Para cada jugador:	
Puntajes acumulados (nuevos)	=
Puntajes acumulados (viejos)	+
Puntajes ronda (nuevos)	

Ronda		Jugador			
					
1	Puntajes acumulados	0	0	0	0
	Puntajes ronda				
	Puntajes acumulados				
2	Puntajes ronda				
	Puntajes acumulados				
	Puntajes ronda				
3	Puntajes acumulados				
	Puntajes ronda				
	Puntajes acumulados				
4	Puntajes ronda				
	Puntajes acumulados				
	Puntajes ronda				
5	Puntajes acumulados				
	Puntajes ronda				
	Puntajes acumulados				



¡A jugar!

- Hacer **grupos de cuatro (4) jugadores**.

¡A jugar!

- Hacer **grupos de cuatro (4) jugadores**.
- Jugar al **1000** (en lugar de 10mil) (si no estamos 10mil años).

¡A jugar!

- Hacer **grupos de cuatro (4) jugadores**.
- Jugar al **1000** (en lugar de 10mil) (si no estamos 10mil años).
- ¿Cuántas **rondas** fueron necesarias para que **al menos un** jugador llegue a **1000**?

PONEMOS EN COMÚN

Guía de supervivencia

1 Entender el problema

Guía de supervivencia

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.

Guía de supervivencia

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)

Guía de supervivencia

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes?

Guía de supervivencia

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes?
- 5 Hacer un boceto de los **pasos a seguir**.

Guía de supervivencia

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes?
- 5 Hacer un boceto de los **pasos a seguir**.
- 6 Planificar el trabajo en la compu.

Guía de supervivencia

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes?
- 5 Hacer un boceto de los **pasos a seguir**.
- 6 Planificar el trabajo en la compu.
- 7 **Programar**, traduciendo la secuencia de pasos a código.

Guía de supervivencia

- 1 Entender el problema
- 2 Identificar cuáles son los **datos** que recibimos.
- 3 Identificar **qué clase de respuesta** debemos dar (¿un número? ¿texto? ¿una lista de valores? ¿varias cosas?)
- 4 **Simplificar** el problema: ¿qué elementos del problema y de los datos son realmente relevantes?
- 5 Hacer un boceto de los **pasos a seguir**.
- 6 Planificar el trabajo en la compu.
- 7 **Programar**, traduciendo la secuencia de pasos a código.
- 8 **Evaluar la solución**: ¿tiene sentido? ¿Cómo nos damos cuenta si cometimos algún error?

Aplicamos la receta al juego

Tenemos una pregunta concreta:

¿Cuántas rondas hacen falta para que **al menos un jugador** llegue a 1000?

Aplicamos la receta al juego

Tenemos una pregunta concreta:

¿Cuántas rondas hacen falta para que **al menos un jugador** llegue a 1000?

- **Datos:** cantidad de jugadores, puntajes acumulados, resultados de cada tirada.

Aplicamos la receta al juego

Tenemos una pregunta concreta:

¿Cuántas rondas hacen falta para que **al menos un jugador** llegue a 1000?

- **Datos:** cantidad de jugadores, puntajes acumulados, resultados de cada tirada.
- **Respuesta:** un número, la cantidad de rondas.

Aplicamos la receta al juego

Tenemos una pregunta concreta:

¿Cuántas rondas hacen falta para que **al menos un jugador** llegue a 1000?

- **Datos:** cantidad de jugadores, puntajes acumulados, resultados de cada tirada.
- **Respuesta:** un número, la cantidad de rondas.
- **Idea:** necesitamos **repetir** las rondas hasta que alguien llegue a 1000.

Nuestra receta

En cada ronda:

- Cada jugador tira su cubilete de dados.

Nuestra receta

En cada ronda:

- 1 Cada jugador tira su cubilete de dados.
- 2 Analizamos los resultados calculando el puntaje.

Nuestra receta

En cada ronda:

- 1 Cada jugador tira su cubilete de dados.
- 2 Analizamos los resultados calculando el puntaje.
- 3 Los registramos actualizando los puntajes acumulados.

Nuestra receta

En cada ronda:

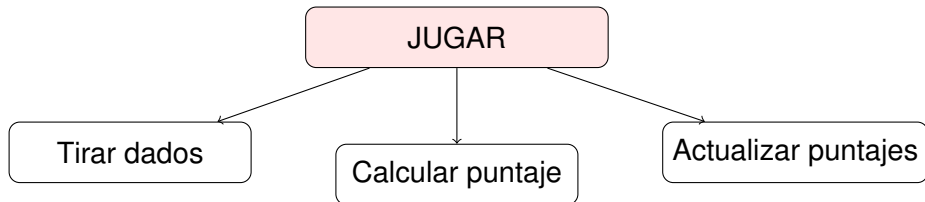
- 1 Cada jugador tira su cubilete de dados.
- 2 Analizamos los resultados calculando el puntaje.
- 3 Los registramos actualizando los puntajes acumulados.
- 4 Actualizamos el contador de rondas.

Nuestra receta

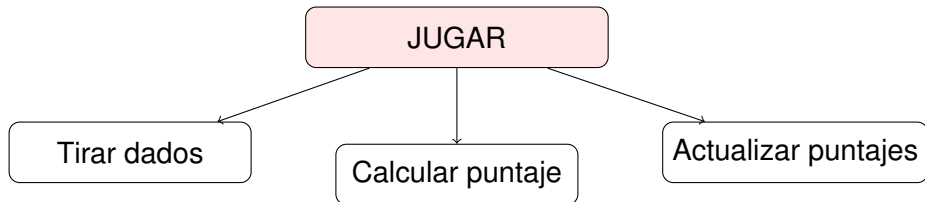
En cada ronda:

- 1 Cada jugador tira su cubilete de dados.
- 2 Analizamos los resultados calculando el puntaje.
- 3 Los registramos actualizando los puntajes acumulados.
- 4 Actualizamos el contador de rondas.
- 5 Vemos si el juego debe continuar o no (¿Alguien llegó a 1000?).

Pero la receta todavía es grande...

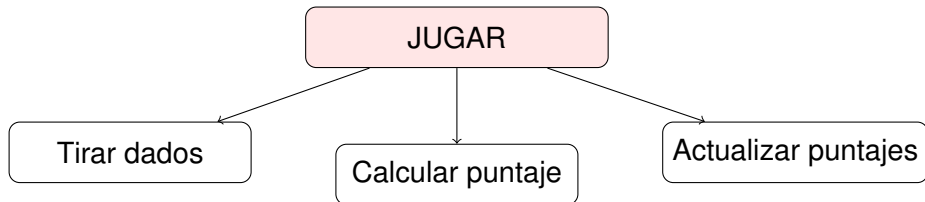


Pero la receta todavía es grande...



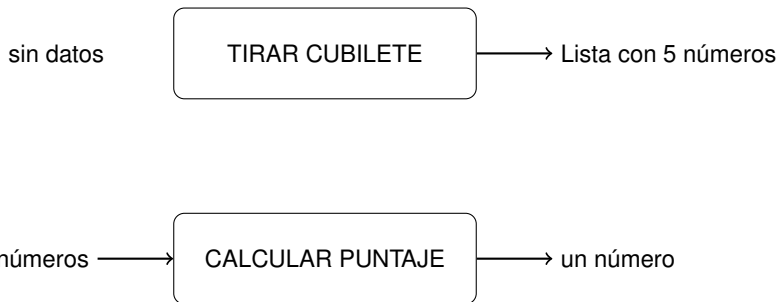
- Hay partes de la receta que podemos resolver **por separado**.

Pero la receta todavía es grande...

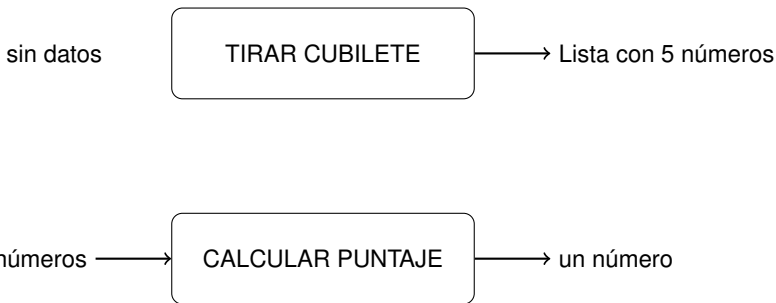


- Hay partes de la receta que podemos resolver **por separado**.
- Y después podemos **combinar** esos resultados para solucionar nuestro problema.

Pensamos las funciones como cajas



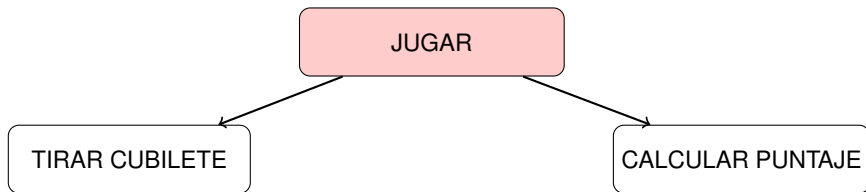
Pensamos las funciones como cajas



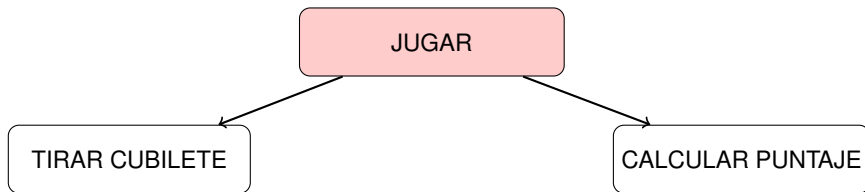
Idea importante

Una función puede ser pensada por lo que **recibe**, lo que **hace** y lo que **devuelve**, sin necesidad de saber todavía cómo está implementada.

¿Quién coordina todo esto?

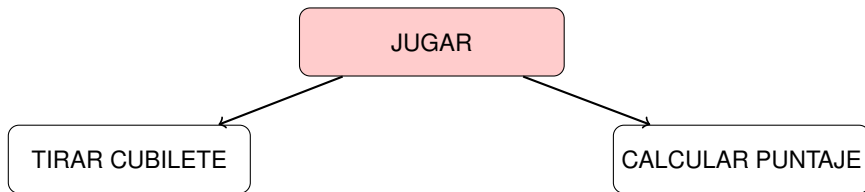


¿Quién coordina todo esto?



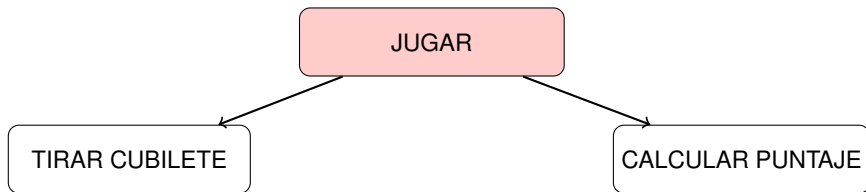
- JUGAR no necesita saber cómo se **tiran** los dados.

¿Quién coordina todo esto?



- JUGAR no necesita saber cómo se **tiran** los dados.
- Tampoco necesita saber cómo se **calcula** el puntaje.

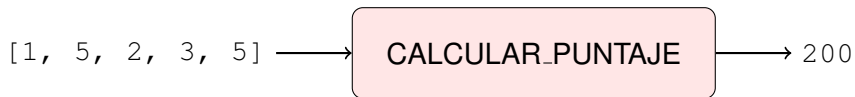
¿Quién coordina todo esto?



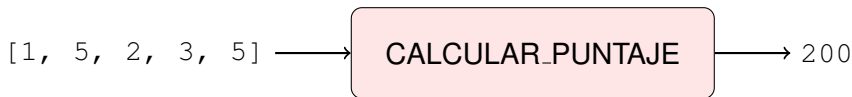
- JUGAR no necesita saber cómo se **tiran** los dados.
- Tampoco necesita saber cómo se **calcula** el puntaje.
- Solo necesita saber **qué función llamar** y qué **resultado esperar**.

PENSANDO EL PUNTAJE

Hacemos zoom en `calcular_puntaje`



Hacemos zoom en `calcular_puntaje`



¿Cómo sabe la función que debe devolver 200?

La función tiene que **analizar** la tirada y **aplicar** las reglas del juego.

¿Qué tenemos que averiguar?

Para calcular el puntaje necesitamos responder preguntas sobre la tirada:

- ¿Cuántos **unos** hay?

¿Qué tenemos que averiguar?

Para calcular el puntaje necesitamos responder preguntas sobre la tirada:

- ¿Cuántos **unos** hay?
- ¿Cuántos **cincos** hay?

¿Qué tenemos que averiguar?

Para calcular el puntaje necesitamos responder preguntas sobre la tirada:

- ¿Cuántos **unos** hay?
- ¿Cuántos **cincos** hay?
- ¿Hay un combo de **3 unos**?

¿Qué tenemos que averiguar?

Para calcular el puntaje necesitamos responder preguntas sobre la tirada:

- ¿Cuántos **unos** hay?
- ¿Cuántos **cincos** hay?
- ¿Hay un combo de **3 unos**?
- ¿Hay un combo de **5 unos**?

¿Qué tenemos que averiguar?

Para calcular el puntaje necesitamos responder preguntas sobre la tirada:

- ¿Cuántos **unos** hay?
- ¿Cuántos **cincos** hay?
- ¿Hay un combo de **3 unos**?
- ¿Hay un combo de **5 unos**?
- ¿Hay un combo de **3 cincos**?

¿Qué tenemos que averiguar?

Para calcular el puntaje necesitamos responder preguntas sobre la tirada:

- ¿Cuántos **unos** hay?
- ¿Cuántos **cincos** hay?
- ¿Hay un combo de **3 unos**?
- ¿Hay un combo de **5 unos**?
- ¿Hay un combo de **3 cincos**?
- ¿Hay un combo de **5 cincos**?

¿Qué tenemos que averiguar?

Para calcular el puntaje necesitamos responder preguntas sobre la tirada:

- ¿Cuántos **unos** hay?
- ¿Cuántos **cincos** hay?
- ¿Hay un combo de **3 unos**?
- ¿Hay un combo de **5 unos**?
- ¿Hay un combo de **3 cincos**?
- ¿Hay un combo de **5 cincos**?

¿Qué tenemos que averiguar?

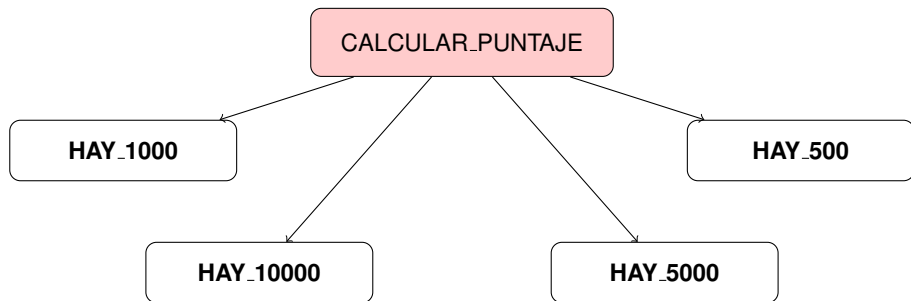
Para calcular el puntaje necesitamos responder preguntas sobre la tirada:

- ¿Cuántos **unos** hay?
- ¿Cuántos **cincos** hay?
- ¿Hay un combo de **3 unos**?
- ¿Hay un combo de **5 unos**?
- ¿Hay un combo de **3 cincos**?
- ¿Hay un combo de **5 cincos**?

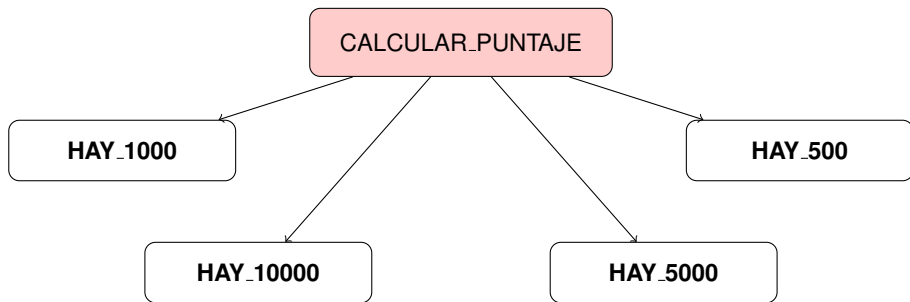
Aparece un nuevo problema

No alcanza con hacer siempre las mismas operaciones: tenemos que **tomar decisiones** dependiendo de lo que encontremos en la tirada.

Una estrategia: separar las preguntas

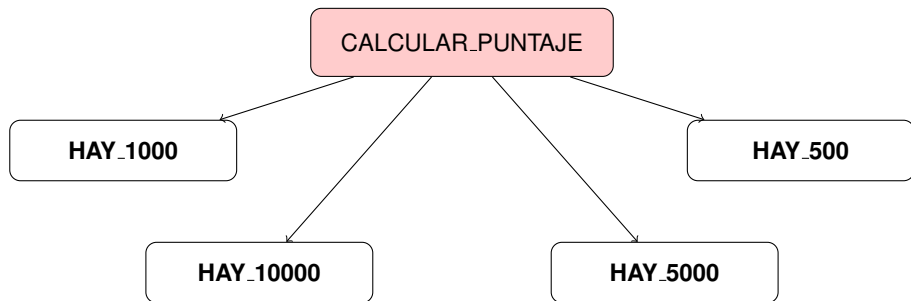


Una estrategia: separar las preguntas



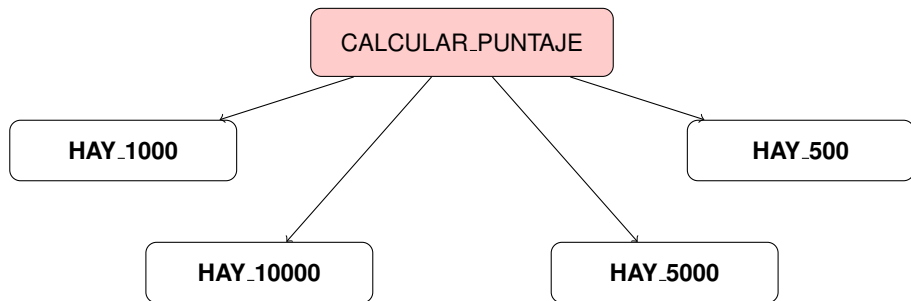
- Cada función responde una pregunta concreta.

Una estrategia: separar las preguntas



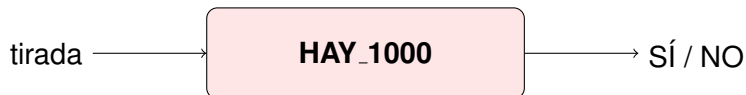
- Cada función responde una pregunta concreta.
- Todas reciben la tirada.

Una estrategia: separar las preguntas

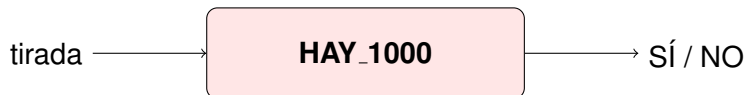


- Cada función responde una pregunta concreta.
- Todas reciben la tirada.
- Y cada una puede devolver simplemente **sí o no**.

Antes de programar: una función que responde una pregunta

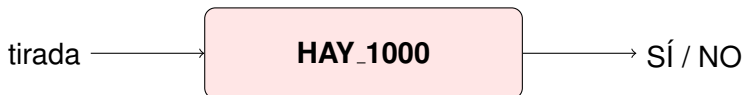


Antes de programar: una función que responde una pregunta



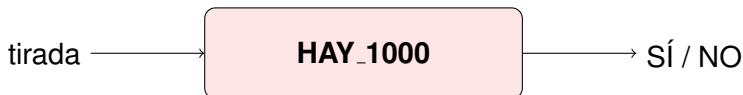
- La pregunta concreta es: **¿hay tres unos?**

Antes de programar: una función que responde una pregunta



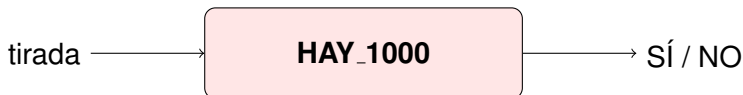
- La pregunta concreta es: **¿hay tres unos?**
- La respuesta solo puede ser una de dos opciones: **sí** o **no**.

Antes de programar: una función que responde una pregunta



- La pregunta concreta es: **¿hay tres unos?**
- La respuesta solo puede ser una de dos opciones: **sí** o **no**.
- En programación esas respuestas son `True` (Verdadero / sí) y `False` (Falso / no).

Antes de programar: una función que responde una pregunta



- La pregunta concreta es: **¿hay tres unos?**
- La respuesta solo puede ser una de dos opciones: **sí** o **no**.
- En programación esas respuestas son `True` (Verdadero / sí) y `False` (Falso / no).
- A `True` y `False` se los conocen como valores booleanos.

Y ahora podemos describir el puntaje

Todavía sin pensar en `Python`, podemos escribir una receta más detallada:

- 1 Calcular el puntaje básico: 100 por cada uno y 50 por cada cinco.

Y ahora podemos describir el puntaje

Todavía sin pensar en `Python`, podemos escribir una receta más detallada:

- 1 Calcular el puntaje básico: 100 por cada uno y 50 por cada cinco.
- 2 Preguntar si hay un combo de cinco unos.

Y ahora podemos describir el puntaje

Todavía sin pensar en `Python`, podemos escribir una receta más detallada:

- 1 Calcular el puntaje básico: 100 por cada uno y 50 por cada cinco.
- 2 Preguntar si hay un combo de cinco unos.
- 3 Si lo hay, agregar 10.000 puntos.

Y ahora podemos describir el puntaje

Todavía sin pensar en `Python`, podemos escribir una receta más detallada:

- 1 Calcular el puntaje básico: 100 por cada uno y 50 por cada cinco.
- 2 Preguntar si hay un combo de cinco unos.
- 3 Si lo hay, agregar 10.000 puntos.
- 4 Preguntar si hay un combo de tres unos.

Y ahora podemos describir el puntaje

Todavía sin pensar en `Python`, podemos escribir una receta más detallada:

- 1 Calcular el puntaje básico: 100 por cada uno y 50 por cada cinco.
- 2 Preguntar si hay un combo de cinco unos.
- 3 Si lo hay, agregar 10.000 puntos.
- 4 Preguntar si hay un combo de tres unos.
- 5 Si lo hay, agregar 1.000 puntos.

Y ahora podemos describir el puntaje

Todavía sin pensar en `Python`, podemos escribir una receta más detallada:

- 1 Calcular el puntaje básico: 100 por cada uno y 50 por cada cinco.
- 2 Preguntar si hay un combo de cinco unos.
- 3 Si lo hay, agregar 10.000 puntos.
- 4 Preguntar si hay un combo de tres unos.
- 5 Si lo hay, agregar 1.000 puntos.
- 6 Repetir la idea para los cincos.

Y ahora podemos describir el puntaje

Todavía sin pensar en `Python`, podemos escribir una receta más detallada:

- 1 Calcular el puntaje básico: 100 por cada uno y 50 por cada cinco.
- 2 Preguntar si hay un combo de cinco unos.
- 3 Si lo hay, agregar 10.000 puntos.
- 4 Preguntar si hay un combo de tres unos.
- 5 Si lo hay, agregar 1.000 puntos.
- 6 Repetir la idea para los cincos.
- 7 Devolver el puntaje total.

Y ahora podemos describir el puntaje

Todavía sin pensar en `Python`, podemos escribir una receta más detallada:

- 1 Calcular el puntaje básico: 100 por cada uno y 50 por cada cinco.
- 2 Preguntar si hay un combo de cinco unos.
- 3 Si lo hay, agregar 10.000 puntos.
- 4 Preguntar si hay un combo de tres unos.
- 5 Si lo hay, agregar 1.000 puntos.
- 6 Repetir la idea para los cincos.
- 7 Devolver el puntaje total.

Y ahora podemos describir el puntaje

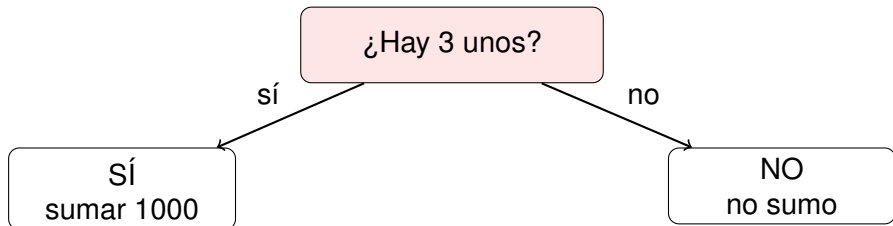
Todavía sin pensar en `Python`, podemos escribir una receta más detallada:

- 1 Calcular el puntaje básico: 100 por cada uno y 50 por cada cinco.
- 2 Preguntar si hay un combo de cinco unos.
- 3 Si lo hay, agregar 10.000 puntos.
- 4 Preguntar si hay un combo de tres unos.
- 5 Si lo hay, agregar 1.000 puntos.
- 6 Repetir la idea para los cincos.
- 7 Devolver el puntaje total.

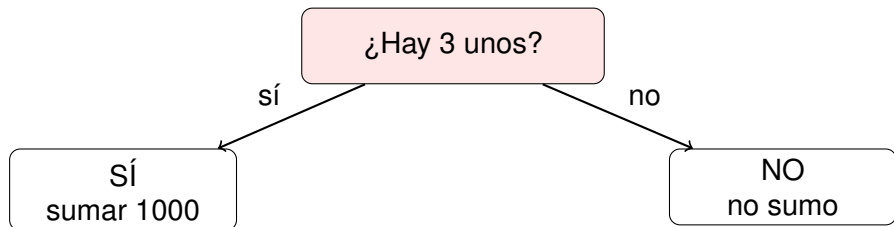
El paso condicional

Hay pasos que dicen: “**hacer algo si pasa algo**”.

Tomar decisiones



Tomar decisiones



Definición

Una **condición** es una pregunta cuya respuesta permite decidir qué hacer. En programación, es una expresión que devuelve un valor booleano (`True/False`).

CONDICIONALES

¿Cómo le decimos “sí” a Python?

Python tiene una **estructura** para ejecutar un bloque de instrucciones **solo cuando una condición es verdadera (True)**.

¿Cómo le decimos “sí” a Python?

Python tiene una **estructura** para ejecutar un bloque de instrucciones **solo cuando una condición es verdadera (True)**.

Condicionales: `if`

Usamos la estructura `if` (*si* en inglés) para **hacer algo** en caso de que una **condición** sea **cierta**.

```
if hay_1000(tirada):  
    puntaje = puntaje + 1000
```

¿Cómo le decimos “sí” a Python?

Python tiene una **estructura** para ejecutar un bloque de instrucciones **solo cuando una condición es verdadera (True)**.

Condicionales: `if`

Usamos la estructura `if` (*si* en inglés) para **hacer algo** en caso de que una **condición** sea **cierta**.

```
if hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- Primero evaluamos la función que devuelve `True` o `False`: **¿hay 1000 en la tirada?**

¿Cómo le decimos “sí” a Python?

Python tiene una **estructura** para ejecutar un bloque de instrucciones **solo cuando una condición es verdadera (True)**.

Condicionales: `if`

Usamos la estructura `if` (*si* en inglés) para **hacer algo** en caso de que una **condición** sea **cierta**.

```
if hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- Primero evaluamos la función que devuelve `True` o `False`: **¿hay 1000 en la tirada?**
- Si la respuesta es `True`, sumamos 1000.

¿Cómo le decimos “sí” a Python?

Python tiene una **estructura** para ejecutar un bloque de instrucciones **solo cuando una condición es verdadera (True)**.

Condicionales: `if`

Usamos la estructura `if` (*si* en inglés) para **hacer algo** en caso de que una **condición** sea **cierta**.

```
if hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- Primero evaluamos la función que devuelve `True` o `False`: **¿hay 1000 en la tirada?**
- Si la respuesta es `True`, sumamos 1000.
- Si la respuesta es `False`, no hacemos esa suma y el programa continúa (en ese ejemplo particular, termina).

Volvemos a mirar la función

Ahora la receta de `calcular_puntaje` empieza a parecerse al programa (aunque sabemos que todavía quedan detalles):

```
puntaje = puntaje_basico(tirada)

if hay_10000(tirada):
    puntaje = puntaje + 10000
if hay_1000(tirada):
    puntaje = puntaje + 1000
if hay_5000(tirada):
    puntaje = puntaje + 5000
if hay_500(tirada):
    puntaje = puntaje + 500
```

Volvemos a mirar la función

Ahora la receta de `calcular_puntaje` empieza a parecerse al programa (aunque sabemos que todavía quedan detalles):

```
puntaje = puntaje_basico(tirada)

if hay_10000(tirada):
    puntaje = puntaje + 10000
if hay_1000(tirada):
    puntaje = puntaje + 1000
if hay_5000(tirada):
    puntaje = puntaje + 5000
if hay_500(tirada):
    puntaje = puntaje + 500
```

Separar el problema en funciones

La dinámica general usa funciones más pequeñas para resolver el problema.

¿Qué pasa si tiramos 5 unos?

Probemos el código anterior con una tirada de **cinco unos**:

[1, 1, 1, 1, 1].

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
if hay_1000(tirada):  
    puntaje = puntaje + 1000
```

¿Qué pasa si tiramos 5 unos?

Probemos el código anterior con una tirada de **cinco unos**:

`[1, 1, 1, 1, 1]`.

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
if hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- `hay_10000(tirada)` es `True`, se ejecuta `puntaje = 0 + 10000 = 10000`.

¿Qué pasa si tiramos 5 unos?

Probemos el código anterior con una tirada de **cinco unos**:

`[1, 1, 1, 1, 1]`.

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
if hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- `hay_10000(tirada)` es **True**, se ejecuta `puntaje = 0 + 10000 = 10000`.
- Pero **también** hay 3 unos entre esos 5, así que `hay_1000(tirada)` **también** es **True**.

¿Qué pasa si tiramos 5 unos?

Probemos el código anterior con una tirada de **cinco unos**:

`[1, 1, 1, 1, 1]`.

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
if hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- `hay_10000(tirada)` es **True**, se ejecuta `puntaje = 0 + 10000 = 10000`.
- Pero **también** hay 3 unos entre esos 5, así que `hay_1000(tirada)` **también** es **True**.
- Y también se ejecuta `puntaje = 10000 + 1000 =` **11000**

¿Qué pasa si tiramos 5 unos?

Probemos el código anterior con una tirada de **cinco unos**:

`[1, 1, 1, 1, 1]`.

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
if hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- `hay_10000(tirada)` es **True**, se ejecuta `puntaje = 0 + 10000 = 10000`.
- Pero **también** hay 3 unos entre esos 5, así que `hay_1000(tirada)` **también** es **True**.
- Y también se ejecuta `puntaje = 10000 + 1000 = 11000`

¿Qué pasa si tiramos 5 unos?

Probemos el código anterior con una tirada de **cinco unos**:

`[1, 1, 1, 1, 1]`.

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
if hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- `hay_10000(tirada)` es `True`, se ejecuta `puntaje = 0 + 10000 = 10000`.
- Pero **también** hay 3 unos entre esos 5, así que `hay_1000(tirada)` **también** es `True`.
- Y también se ejecuta `puntaje = 10000 + 1000 = 11000`

La regla decía: **solo se usa el combo de mayor valor**

Con dos `if` independientes, **sumamos los dos combos** para el mismo número. Necesitamos que, si ya se cumplió la condición de mayor valor, **no se chequeen las siguientes**.

Cuando tenemos que elegir entre dos casos

Regla importante del juego

No se pueden **combinar** dos combos para el mismo número.
Si tenemos 5 unos, usamos el combo de mayor valor: 10.000, no $10.000 + 1.000$.

Cuando tenemos que elegir entre dos casos

Regla importante del juego

No se pueden **combinar** dos combos para el mismo número.
Si tenemos 5 unos, usamos el combo de mayor valor: 10.000, no $10.000 + 1.000$.

Necesitamos expresar algo como:

Si hay 5 unos, hacer una cosa; si no, si hay 3 unos, hacer otra.

Cuando tenemos que elegir entre dos casos

Regla importante del juego

No se pueden **combinar** dos combos para el mismo número.
Si tenemos 5 unos, usamos el combo de mayor valor: 10.000, no $10.000 + 1.000$.

Necesitamos expresar algo como:

Si hay 5 unos, hacer una cosa; si no, si hay 3 unos, hacer otra.

Para eso aparece **elif**.

If, elif y else

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
elif hay_1000(tirada):  
    puntaje = puntaje + 1000
```

If, elif y else

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
elif hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- Se chequea primero la condición de mayor valor.

If, elif y else

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
elif hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- Se chequea primero la condición de mayor valor.
- Si es verdadera, se ejecuta esa rama (por ejemplo, `puntaje = puntaje + 10000`).

If, elif y else

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
elif hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- Se chequea primero la condición de mayor valor.
- Si es verdadera, se ejecuta esa rama (por ejemplo, `puntaje = puntaje + 10000`).
- Las siguientes condiciones ya no se chequean.

If, elif y else

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
elif hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- Se chequea primero la condición de mayor valor.
- Si es verdadera, se ejecuta esa rama (por ejemplo, `puntaje = puntaje + 10000`).
- Las siguientes condiciones ya no se chequean.
- Si la primera condición fue `False`, recién ahí se chequea la segunda.

If, elif y else

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
elif hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- Se chequea primero la condición de mayor valor.
- Si es verdadera, se ejecuta esa rama (por ejemplo, `puntaje = puntaje + 10000`).
- Las siguientes condiciones ya no se chequean.
- Si la primera condición fue `False`, recién ahí se chequea la segunda.
- Se pueden agregar tantos `elif` como sea necesario.

If, elif y else

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
elif hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- Se chequea primero la condición de mayor valor.
- Si es verdadera, se ejecuta esa rama (por ejemplo, `puntaje = puntaje + 10000`).
- Las siguientes condiciones ya no se chequean.
- Si la primera condición fue `False`, recién ahí se chequea la segunda.
- Se pueden agregar tantos `elif` como sea necesario.

If, elif y else

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
elif hay_1000(tirada):  
    puntaje = puntaje + 1000
```

- Se chequea primero la condición de mayor valor.
- Si es verdadera, se ejecuta esa rama (por ejemplo, `puntaje = puntaje + 10000`).
- Las siguientes condiciones ya no se chequean.
- Si la primera condición fue `False`, recién ahí se chequea la segunda.
- Se pueden agregar tantos `elif` como sea necesario.

Cumpliendo las reglas de juego

Este mecanismo permite respetar la regla del juego: **solo usamos el combo de mayor valor para los unos.**

¿Y si queremos hacer algo cuando no se cumple?

Podemos agregar **else**:

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
elif hay_1000(tirada):  
    puntaje = puntaje + 1000  
else:  
    # no hay combo de unos
```

¿Y si queremos hacer algo cuando no se cumple?

Podemos agregar **else**:

```
if hay_10000(tirada):  
    puntaje = puntaje + 10000  
elif hay_1000(tirada):  
    puntaje = puntaje + 1000  
else:  
    # no hay combo de unos
```

else

Es la opción para cuando todos los casos anteriores no se cumplieron. Se imaginan que solo puede haber un **único** **else**.

COMPARACIONES Y CONDICIONES

¿De dónde salen las condiciones?

- Hasta ahora usamos funciones que ya nos daban una respuesta `True` o `False`.

¿De dónde salen las condiciones?

- Hasta ahora usamos funciones que ya nos daban una respuesta `True` o `False`.
- Pero muchas veces necesitamos construir la condición nosotros mismos:

¿De dónde salen las condiciones?

- Hasta ahora usamos funciones que ya nos daban una respuesta `True` o `False`.
- Pero muchas veces necesitamos construir la condición nosotros mismos:
 - ¿La cantidad de unos es **igual a 3**?

¿De dónde salen las condiciones?

- Hasta ahora usamos funciones que ya nos daban una respuesta `True` o `False`.
- Pero muchas veces necesitamos construir la condición nosotros mismos:
 - ¿La cantidad de unos es **igual a 3**?
 - ¿El puntaje es **mayor o igual a 1000**?

¿De dónde salen las condiciones?

- Hasta ahora usamos funciones que ya nos daban una respuesta `True` o `False`.
- Pero muchas veces necesitamos construir la condición nosotros mismos:
 - ¿La cantidad de unos es **igual a 3**?
 - ¿El puntaje es **mayor o igual a 1000**?
 - ¿La cantidad de cincos es **distinta de 0**?

¿De dónde salen las condiciones?

- Hasta ahora usamos funciones que ya nos daban una respuesta `True` o `False`.
- Pero muchas veces necesitamos construir la condición nosotros mismos:
 - ¿La cantidad de unos es **igual a 3**?
 - ¿El puntaje es **mayor o igual a 1000**?
 - ¿La cantidad de cincos es **distinta de 0**?

¿De dónde salen las condiciones?

- Hasta ahora usamos funciones que ya nos daban una respuesta `True` o `False`.
- Pero muchas veces necesitamos construir la condición nosotros mismos:
 - ¿La cantidad de unos es **igual a 3**?
 - ¿El puntaje es **mayor o igual a 1000**?
 - ¿La cantidad de cincos es **distinta de 0**?
- Para eso usamos **comparaciones**.

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor
 - `<=` menor o igual

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor
 - `<=` menor o igual
 - `>` mayor

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor
 - `<=` menor o igual
 - `>` mayor
 - `>=` mayor o igual

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor
 - `<=` menor o igual
 - `>` mayor
 - `>=` mayor o igual
 - `==` igual

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor
 - `<=` menor o igual
 - `>` mayor
 - `>=` mayor o igual
 - `==` igual
 - `!=` distinto

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor
 - `<=` menor o igual
 - `>` mayor
 - `>=` mayor o igual
 - `==` igual
 - `!=` distinto
- También se pueden **combinar** distintas condiciones utilizando los operadores lógicos:

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor
 - `<=` menor o igual
 - `>` mayor
 - `>=` mayor o igual
 - `==` igual
 - `!=` distinto
- También se pueden **combinar** distintas condiciones utilizando los operadores lógicos:
 - **not** negación, si se aplica a `True`, da `False` y a la inversa.

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor
 - `<=` menor o igual
 - `>` mayor
 - `>=` mayor o igual
 - `==` igual
 - `!=` distinto
- También se pueden **combinar** distintas condiciones utilizando los operadores lógicos:
 - **not** negación, si se aplica a `True`, da `False` y a la inversa.
 - **and** se usa `x and y`. Solo da `True` cuando `x` e `y` son `True`.

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor
 - `<=` menor o igual
 - `>` mayor
 - `>=` mayor o igual
 - `==` igual
 - `!=` distinto
- También se pueden **combinar** distintas condiciones utilizando los operadores lógicos:
 - **not** negación, si se aplica a `True`, da `False` y a la inversa.
 - **and** se usa `x and y`. Solo da `True` cuando `x` e `y` son `True`.
 - **or** se usa `x or y`. Da `True` cuando alguna de las dos (o las dos) es `True`.

Comparaciones y condiciones

- El resultado de una comparación puede ser: `True` o `False`.
- Se pueden realizar distintas comparaciones:
 - `<` menor
 - `<=` menor o igual
 - `>` mayor
 - `>=` mayor o igual
 - `==` igual
 - `!=` distinto
- También se pueden **combinar** distintas condiciones utilizando los operadores lógicos:
 - **not** negación, si se aplica a `True`, da `False` y a la inversa.
 - **and** se usa `x and y`. Solo da `True` cuando `x` e `y` son `True`.
 - **or** se usa `x or y`. Da `True` cuando alguna de las dos (o las dos) es `True`.
- Las comparaciones se pueden usar como condiciones del `if` y del `while`.

Una condición puede ser una comparación

```
cantidad_unos = 3

if cantidad_unos == 3:
    puntaje = puntaje + 1000
```

Una condición puede ser una comparación

```
cantidad_unos = 3
```

```
if cantidad_unos == 3:  
    puntaje = puntaje + 1000
```

- `cantidad_unos == 3` es una pregunta.

Una condición puede ser una comparación

```
cantidad_unos = 3

if cantidad_unos == 3:
    puntaje = puntaje + 1000
```

- `cantidad_unos == 3` es una pregunta.
- La respuesta es `True` o `False`.

Una condición puede ser una comparación

```
cantidad_unos = 3

if cantidad_unos == 3:
    puntaje = puntaje + 1000
```

- `cantidad_unos == 3` es una pregunta.
- La respuesta es `True` o `False`.
- El `if` usa esa respuesta para decidir qué hacer.

También podemos combinar condiciones

A veces una decisión depende de más de una pregunta.

También podemos combinar condiciones

A veces una decisión depende de más de una pregunta.

- `and`: las dos condiciones deben ser verdaderas.

También podemos combinar condiciones

A veces una decisión depende de más de una pregunta.

- `and`: las dos condiciones deben ser verdaderas.
- `or`: alcanza con que una sea verdadera.

También podemos combinar condiciones

A veces una decisión depende de más de una pregunta.

- `and`: las dos condiciones deben ser verdaderas.
- `or`: alcanza con que una sea verdadera.
- `not`: invierte la respuesta.

También podemos combinar condiciones

A veces una decisión depende de más de una pregunta.

- `and`: las dos condiciones deben ser verdaderas.
- `or`: alcanza con que una sea verdadera.
- `not`: invierte la respuesta.

También podemos combinar condiciones

A veces una decisión depende de más de una pregunta.

- `and`: las dos condiciones deben ser verdaderas.
- `or`: alcanza con que una sea verdadera.
- `not`: invierte la respuesta.

Por ejemplo:

```
cantidad_unos > 0 and cantidad_cincos > 0
```

También podemos combinar condiciones

A veces una decisión depende de más de una pregunta.

- `and`: las dos condiciones deben ser verdaderas.
- `or`: alcanza con que una sea verdadera.
- `not`: invierte la respuesta.

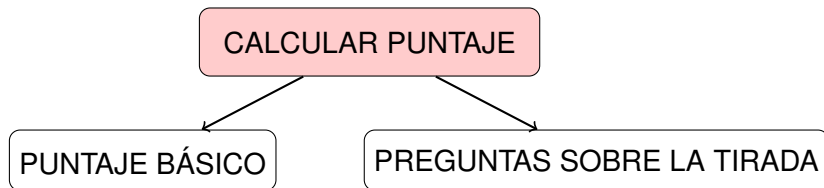
Por ejemplo:

```
cantidad_unos > 0 and cantidad_cincos > 0
```

La condición es verdadera cuando hay al menos un uno **y** al menos un cinco.

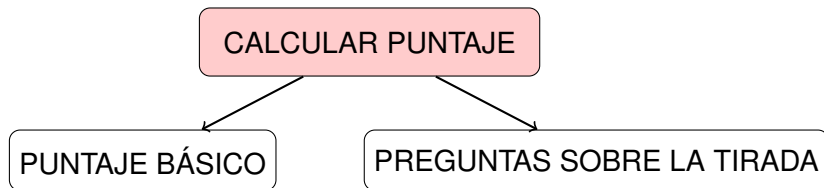
Volvemos al problema original de obtener el puntaje de una tirada

Tenemos una forma de organizar la solución:



Volvemos al problema original de obtener el puntaje de una tirada

Tenemos una forma de organizar la solución:



Diseño de la solución

Las preguntas pueden ser funciones que devuelven `True/False`, y `if/elif/else` nos permiten decidir qué hacer con esas respuestas.

Ejercicio

- Supongamos que existen las funciones:
 - `hay_10000()`
 - `hay_1000()`
 - `hay_5000()`
 - `hay_500()`

Ejercicio

- Supongamos que existen las funciones:
 - `hay_10000()`
 - `hay_1000()`
 - `hay_5000()`
 - `hay_500()`
- Cada una recibe la lista de dados (cinco números, la tirada) y devuelve un valor booleano: `True` o `False`.

Ejercicio

- Supongamos que existen las funciones:
 - `hay_10000()`
 - `hay_1000()`
 - `hay_5000()`
 - `hay_500()`
- Cada una recibe la lista de dados (cinco números, la tirada) y devuelve un valor booleano: `True` o `False`.
- Escribir la función `calcula_puntaje` que recibe la lista de dados y devuelve el puntaje total, respetando las reglas de los combos.

Ejercicio

- Supongamos que existen las funciones:
 - `hay_10000()`
 - `hay_1000()`
 - `hay_5000()`
 - `hay_500()`
- Cada una recibe la lista de dados (cinco números, la tirada) y devuelve un valor booleano: `True` o `False`.
- Escribir la función `calcula_puntaje` que recibe la lista de dados y devuelve el puntaje total, respetando las reglas de los combos.

¡A trabajar!

Para llevarnos hoy

- Una receta de resolución nos ayuda a pasar del problema a una estrategia.

Para llevarnos hoy

- Una receta de resolución nos ayuda a pasar del problema a una estrategia.
- Una estrategia compleja puede dividirse en **funciones** más pequeñas.

Para llevarnos hoy

- Una receta de resolución nos ayuda a pasar del problema a una estrategia.
- Una estrategia compleja puede dividirse en **funciones** más pequeñas.
- Las funciones pueden **combinarse** sin conocer sus detalles internos.

Para llevarnos hoy

- Una receta de resolución nos ayuda a pasar del problema a una estrategia.
- Una estrategia compleja puede dividirse en **funciones** más pequeñas.
- Las funciones pueden **combinarse** sin conocer sus detalles internos.
- Algunas funciones necesitan responder preguntas: **True / False**.

Para llevarnos hoy

- Una receta de resolución nos ayuda a pasar del problema a una estrategia.
- Una estrategia compleja puede dividirse en **funciones** más pequeñas.
- Las funciones pueden **combinarse** sin conocer sus detalles internos.
- Algunas funciones necesitan responder preguntas: **True / False**.
- Las **condiciones** nos permiten tomar decisiones.

Para llevarnos hoy

- Una receta de resolución nos ayuda a pasar del problema a una estrategia.
- Una estrategia compleja puede dividirse en **funciones** más pequeñas.
- Las funciones pueden **combinarse** sin conocer sus detalles internos.
- Algunas funciones necesitan responder preguntas: **True / False**.
- Las **condiciones** nos permiten tomar decisiones.
- En `Python`, usamos `if`, `elif` y `else` para expresar esas decisiones.

Para llevarnos hoy

- Una receta de resolución nos ayuda a pasar del problema a una estrategia.
- Una estrategia compleja puede dividirse en **funciones** más pequeñas.
- Las funciones pueden **combinarse** sin conocer sus detalles internos.
- Algunas funciones necesitan responder preguntas: **True / False**.
- Las **condiciones** nos permiten tomar decisiones.
- En `Python`, usamos `if`, `elif` y `else` para expresar esas decisiones.
- Las **comparaciones** nos permiten construir condiciones.

Esto recién comienza. . .

Terminamos. . . ¡sólo por hoy!